

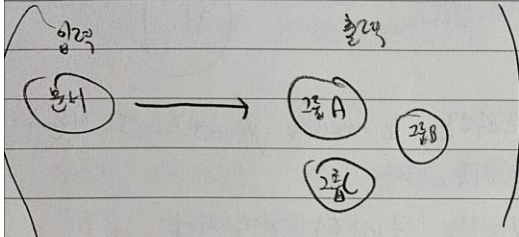
5장 텍스트 클러스터링 / Topic Modeling

Date

테스트 클러스터링: 텍스트를 내용, 의미, 관계 기반의 비슷한 것끼리 그룹으로 묶는 것.

- 구조적이지 않은 대용량 텍스트를 효과적으로 분류

- EDA (탐색적 데이터 분석) 도무늬



- 이상치 찾기, 레이블을 주든 않든, 레이블이 잘못 붙여진 데이터 찾기 등에 활용

Topic modeling: 대형 텍스트 데이터에서 추상적인 토픽 찾기

Car
day
keyword: cars

Soccer
keyword: soccer, sports, ...

Pizza
keyword: pizza, food

일반적인 pipeline

① 임베딩 모델

! 입력 문서를 임베딩으로 변환

② 차원 축소 모델

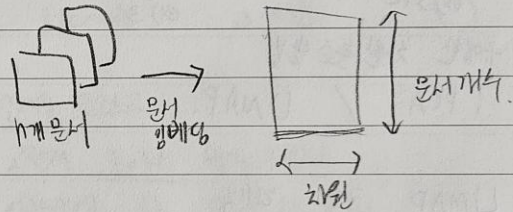
! 임베딩 차원 줄임,

③ 클러스터링 모델

! 의미가 비슷한 문서 그룹 찾음

5.2.1 문서 임베딩

- 텍스트 data → 임베딩으로 바꾸는 것



- 의미 유사도 작업에 최적화된 임베딩 모델 생성해야 하는 것이 중요.

- MTEB 리더보드 사용

- 매우 작은 임베딩 모델이 필요하기도 함

- sentence transformers / all-mpnet-base-v2

- 출력 예시) (449.49, 384)
문서 개수 차원

5.2 텍스트 클러스터링 pipeline

텍스트 클러스터링 사용

→ 익숙하거나 익숙하지 않은 데이터에서 패턴 찾음

- 작업의 복잡도를 직관적으로 이해 도모

- EDA 신속하게 수행함

- 그래프 기반 신경망

- 심플한 기반 클러스터링 방법

5.2.2 임베딩 차원 축소

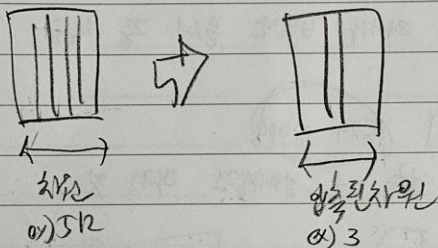
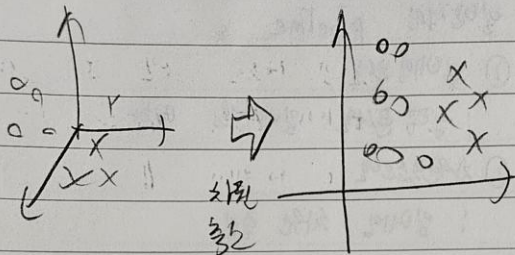
- 차원 수 증가 → 가용한 값의 개수 지수적으로 증가

→ 해당 데이터에서 의미있는 클러스터 찾기가 어렵다.

- 차원 축소 기법 (특히) 고차원 데이터

전역 구조 (global structure) 를 보존하는 저차원 표현 찾기

ex)



- 다양한 차원 축소 방법

: PCA / UMAP.

- UMAP 코드 해석

n-components 매개변수) 저차원 공간의 크기 결정, 일반적으로 5~10 사이의 값이 고차원 전역 구조를 포착하는데 잘 맞음.

min-dist 매개변수)

임베딩된 포인트 사이의 최소 거리.

일반적으로는 이 값을 0으로 지정.

→ 조밀한 클러스터 생성.

유클리드 기반 지표는 고차원 data

다루기 어렵다 → metric 매개변수를

'cosine' 으로 지정

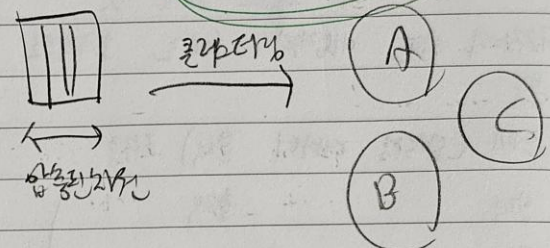
UMAP에서 random-state 지정)

실행할 때마다 같은 결과 재현

but. 병렬 실행 안되어

동일한 결과

5.2.3. 클러스터링 알고리즘



- 일반적으로 k-means / 센트로이드 기반 알고리즘 사용.

→ 생성한 클러스터 개수 지정

→ 클러스터 개수 사전에 알지 X

- 밀도기반 알고리즘

→ 자유롭게 클러스터 개수 결정

→ 모든 데이터 포인트 클러스터에

할당 X

→ HDBSCAN

→ DBSCAN의 계층형 버전

→ 클러스터 개수 지정 X.

→ 조밀한 클러스터 찾을 수 있다.

→ 밀도기반 방법이지만,

→ 이상치 감지

! 어떤 클러스터에 할당 X. 데이터 무시.

5.2.4. 클러스터 평가

(클러스터에 할당된 문서)를 직접 확인하여 검사

ex) (클러스터 0)에서 몇개의 문서 할당 가능.

- 예제 코드는
- 비역한 문서 즉 (클러스터)에서 포함함.
 - 클러스터 결과 시각화
 - matplotlib
 - ↳ 주로 클러스터 색깔로 표현
 - ↳ but 모든 같은 색깔이면 같은 클러스터X
 - (일지 개수의 색깔을 순차에 따른)

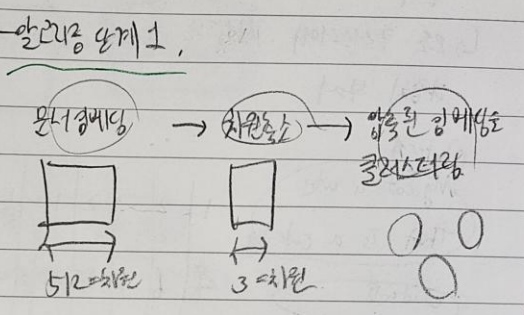
5.3.1 BERTopic

- BERTopic, 의미적으로 유사한 text 클러스터

↳ 다양한 종류의 토픽 표현을 주는 topic modeling 기법.

5.3 Topic modeling.

- 텍스트 클러스터링은 다채로운 문서집합에 내재된 구조를 찾는 데 강력함.
- ex) 수업을 클러스터 조사
- ↳ 이 클러스터의 Topic = 'sign language'



- Topic modeling

↳ 토픽의 의미를 가장 잘 나타내거나 잘 포착하는 keyword 나 구를 찾음.

ex)

알려짐 양제 2.

고전적 BoW 방식 사용

이체제에 있는 단어의 문도 반영.

ex) BoW = 문서에 나타나는 각 단어의 횟수를 카운트

ex) 문서1 →

that	is	a	cat
------	----	---	-----

문서2 →

that	is	a	cat
------	----	---	-----

- LDA

↳ 각 토픽이 많은 문서의 이체제에 있는 단어의 확률분포로 표현됨 가장

↳ BoW 기법 사용.

↳ 단어와 구의 맥락이나 의미 고려X

↳ transformer 기반 임베딩의 의미적 유사도/의미 유사도

가능 포함,

- 결론

BoW는 문서를 표현.

이를 해결하기 위해 단어의 빈도를 '클러스터' 기반으로 계산

ex) LDA

LDA	BERTopic
① 문서 단위 단어 확률분포	① HDBSCAN으로 클러스터링
↓	② 클러스터 단위의 확률분포 (BoW 기반)
각 랜덤 추측 반복	③ 클러스터별 CTFIT
연산	→ 클러스터별 의미적 유사도 계산
토픽을 word에 할당.	

점 2.

the, I 같은 '불용어' 같은건

실제 문서에 의미가 거의 없다는 문제

↳ 클러스터에 의미있는 단어에 높은 가중치 부여

↳ 모든 클러스터에 사용되는 단어에 낮은 가중치 부여

Cluster A

Any cat is cute

That is a cat dog

Cluster B

Cluster C

1	2	1	2	1	1
4	6	3	18	0	50
4	1	3	2	0	30

that is a cat dog dog dog

C-TP

같은 cluster 안에서, 해당 단어가 얼마나 자주 나오는지

IDF

$\log \left(\frac{\text{그 단어의 등장빈도 (클러스터링)}}{\text{다른 클러스터에서 나타나는 그 단어의 총 빈도 수}} \right)$

다음 식) C-TP-IDF

$$= \frac{f_{tc}}{n_c} \cdot \log \left(1 + \frac{N}{f_t} \right)$$

f_{tc} : 도큐먼트 C 이니 단어 t의 등장빈도

n_c : 도큐먼트 C 내 전체 단어의 총빈도

f_t : 전체 도큐먼트 (문서) 이니 단어 t가 등장한 총 빈도 수

N : 전체 도큐먼트 수

32)

Cont Vectorizer

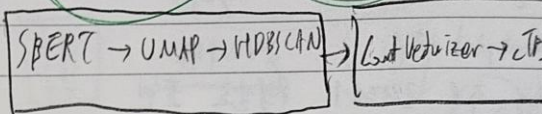
C-TP-IDF

1	2	1	2
4	6	3	18
4	1	3	2

$(C-TP) \cdot \log \left(\frac{N}{f_t} \right)$
A: 등장빈도
C: 총 빈도 수

Smoothing

토픽 클러스터링 + 토픽 표현

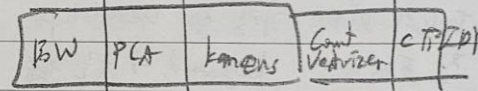
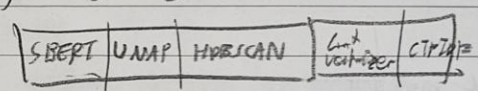


↳ Topic clustering & Topic 표현 두 단계가 서로 독립적.

↳ 파이프라인의 각 구성요소 모듈화에 good

↳ 토픽 표현을 fine tuning 하기 위한 좋은 출발점.

4) 모델을 만들 수 있다.



32) get-topic-info () 메서드

: 발견된 토픽에 대한 간단한 정보 제공

get-topic () 함수

: 개별 토픽 조사 및 가장 잘 표현하는 키워드 찾기

키워드 찾기

find-topics ()

: 검색어 기반 토픽 찾기

5.3.2 Reranker

- 기존 BERTopic의 단점) 하이브리드 구조로 X

- 해결책) 1) Bow로 먼저

바르게 표현 생성 (앞에서 한거)

2) Reranker = 임베딩 모델

= 표현 모델 훈련기 사용

(강력하고 느림)

→ 의미론적 관련도

즉, 같은 토픽 안에서
이 축을 본도 타 단어의 순위를
ReRanker는 재배치.

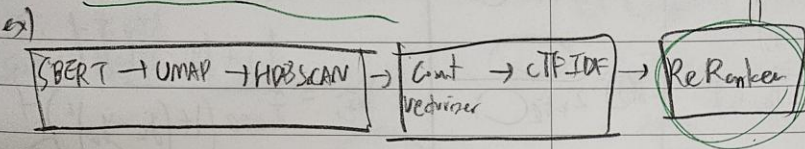
→ 의미론적 구조 고려 ①

ex)

키워드 (인상, 감정, 기억, 의의) → 정제된 키워드 (이전 항목 불포함)

토픽 표현의 최적화된 토픽 개수만큼 수행
ex) 각 ReRanker 블록은 각 토픽에 1번씩만

representation model



ReRanker = Representation 블록 중개

종류 1. KEYBERT Inspired

① 문서의 (C-TP-IDF) 지정지정값 2차

토픽의 (C-TP-IDF) 값 사이의 유사도 기반

→ 토픽이나 대표적인 문서들을

↓ 임베딩 & 토픽 계산 ①

- ② 키워드의 임베딩, 군집

- ③ ①, ②의 코사인 유사도 계산

①의 keyword 순위 재배치

종류 3. LLM 사용

- 모든 문서의 토픽 식별

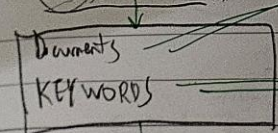
토픽을 위한 label 생성하는 것

- 키워드 생성 X 순위 재배치 X

- 이전에 생성한 키워드 + 대표적 문서 기반

짧은 label 생성

ex) 키워드 + 문서



키워드 생성가 label 생성

토픽을 가장 잘 표현하는
(ex) 전연령의 4차) 외의 생식

토픽을 가장하는
키워드

→ 해당 토픽과 C-TP-IDF 값 유사도가
가장 높은 문서를 선택

C-TP-IDF나 이를 Representation Block
생성 가능

→ 토픽이나 LLM을 한번씩만 쓰면 된다

ibis

종류 2. MMR

- 토픽 표현 다양화 가능

- Summaries, Summary 이런 중복 표현

제거 가능

- 서로 다른 키워드의 집합 (문헌 문서의 양면)
찾으려고 노력함

- 후보 키워드 집합을 임베딩하고,
가장 짧은 키워드를 반복적으로 제거해 주

- 키워드나 문서 다 합쳐서 키워드 나타내는
아래변수 저장 필요

- 중복된 단어 제거 ex. 토픽 표현에 새롭게 키워드는 다

ex) 원본 키워드	ReRanker 사용
medical clinical patient	nlp ehr biomedical... <u>KEYBERT Inspired</u>
...	clinical biomedical healthcare... <u>MMR</u>
	Improved Representation learning for Biomedical... <u>LLM</u>